

GROUP 40
PERIODIC TASK LAYER
(PROJECT #2)

GENERAL OVERVIEW OF THE REPOSITORY

TESTING CONFIGURATION FILES:

- /MAIN.C (DEFAULT)
- /JITTER_RELEASE_TEST.C
- /DEADLINE_MISS_TEST.C
- /SKIP_TEST.C
- /KILL_TEST.C
- /CATCH_UP_TEST.C
- /GRAPHIC_TEST.C

GRAPHICAL INTERFACE AND TESTING FRAMEWORK FILES:

- /MASTEROFPUPPETS.PY
- /MASTEROFPUPPETS2.PY
- /MASTEROFPUPPETSGr.PY

PERIODIC TASK LAYER FILES:

- /PTL.C
- /PTL.H

TRACING UTILITIES FILES:

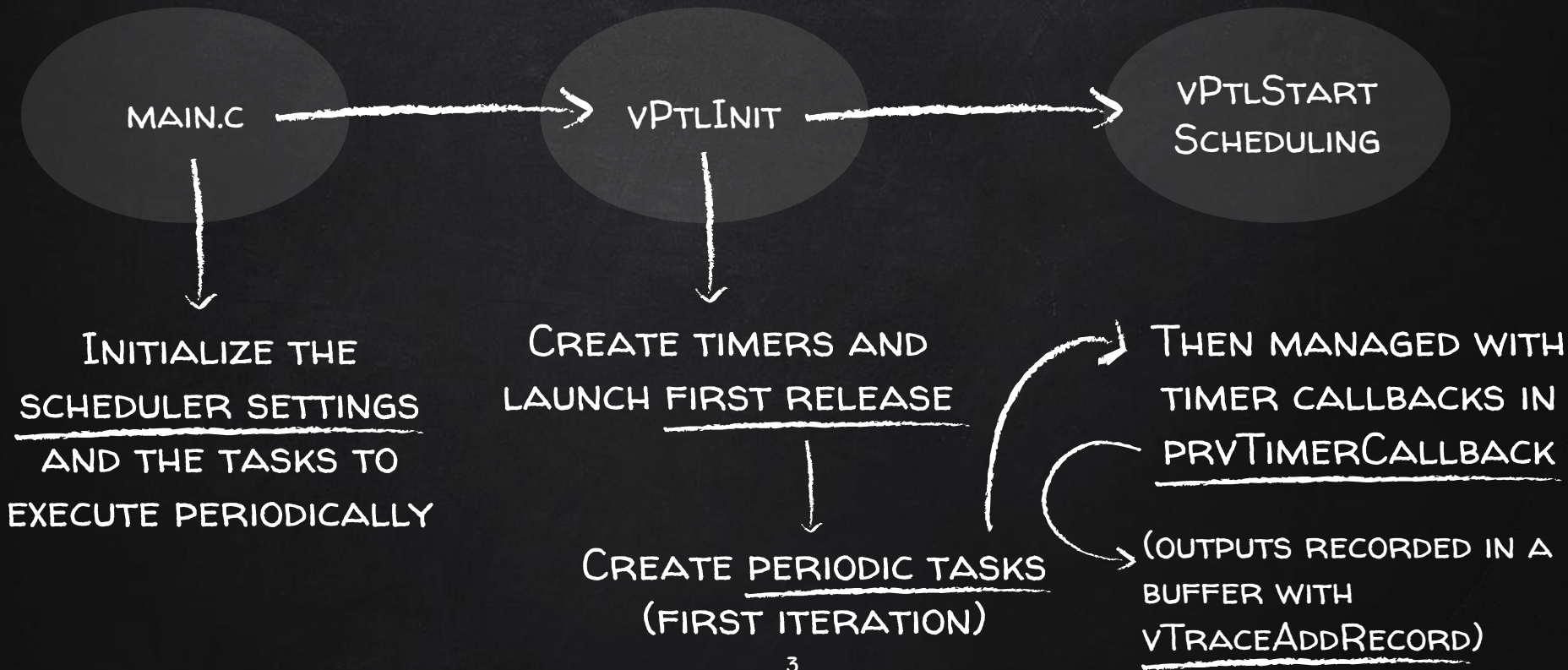
- /TRACE.C
- /TRACE.H

OTHER FILES:

- /MAKEFILE
- /FREERTOSCONFIG.H



THE OVERALL PROCESS FLOW





PTL.H [1]



```
/* POLICY TYPE FOR OVERRUN */  
typedef enum  
{  
    PTL_POLICY_SKIP,  
    PTL_POLICY_KILL,  
    PTL_POLICY_CATCH_UP  
} OverrunPolicy_t;
```

```
/* PTL statistics and behavior */  
typedef struct  
{  
    OverrunPolicy_t ePolicy;  
    bool xTraceEnabled;  
    PeriodicTask_t xActiveTasks [ ptlMAX_PERIODIC_TASKS_NUMBER ];  
    uint8_t ucNumTasks;  
} PTLConfigs_t;
```

```
/* PERIODIC TASK TYPE */  
typedef struct  
{  
    const char *pcTaskName;  
    TaskFunction_t xEntryFunction;  
    void *pxArgument;  
    uint16_t usStackSize;  
    UBaseType_t xPriority;  
    TickType_t xPeriod;  
    TickType_t xDeadline;  
} PeriodicTask_t;
```



PTL.H [2]



```
void vPtlInit( PTLConfigs_t *pxPtlConfigsToSet )
{
    uint8_t ucIndex;

    xGlobalCfg = *pxPtlConfigsToSet;

    for ( ucIndex = 0; ucIndex < pxPtlConfigsToSet->ucNumTasks; ucIndex++ )
    {
        /* Reference: assign tcb from here on, assign to global vector */
        PeriodicTaskTCB_t *pxTcb = &xCurrentActiveTasks[ ucIndex ];
        /* Assigning parameters for the periodic task (periodic_task_t) */
        pxTcb->xPeriodicTask = pxPtlConfigsToSet->xActiveTasks[ ucIndex ];
        /* To make deadline optional */
        if ( pxTcb->xPeriodicTask.xDeadline == 0 )
            pxTcb->xPeriodicTask.xDeadline = pxTcb->xPeriodicTask.xPeriod;

        pxTcb->xJobRunning = false; pxTcb->xJobMissed = false;
        pxTcb->xJobSkipped = false; pxTcb->ulOverrunCount = 0;
        pxTcb->xReleaseTime = 0; pxTcb->ulReleaseNumber = 0;
        pxTcb->xTaskHandle = NULL; ucIndexVect[ ucIndex ] = ucIndex;
    }
}
```

■ ■ ■



PTL.H [3]



```

    ...

    /* Creating the timer for each task */
    pxTcb->xTaskTimer = xTimerCreate(
        pxTcb->xPeriodicTask.pcTaskName,
        pxTcb->xPeriodicTask.xPeriod,
        pdTRUE,
        ( void * )&ucIndexVect[ ucIndex ],
        prvTimerCallback
    );

    /* Timer will start right along with the scheduler */
    xTimerStart( pxTcb->xTaskTimer, 0 );

    prvFirstReleases();
}

void vPtlStartsScheduling( void )
{
    vTaskStartScheduler();
}
```



PTL.C [1]



```
static void prvFirstReleases( void )
{
    uint8_t ucIndex;

    for ( ucIndex = 0; ucIndex < xGlobalCfg.ucNumTasks; ucIndex++ )
    {
        PeriodicTaskTCB_t *pxTcb = &xCurrentActiveTasks[ ucIndex ];
        xTaskCreate(
            prvPTLTaskWrapper,
            pxTcb->xPeriodicTask.pcTaskName,
            pxTcb->xPeriodicTask.usStackSize,
            ( void * )pxTcb,
            pxTcb->xPeriodicTask.xPriority,
            &pxTcb->xTaskHandle
        );
        vTraceAddRecord(/* release */);
    }
}
```



PTL.C [2]



```
static void prvPTLTaskWrapper( void *pxParam )
{
    PeriodicTaskTCB_t *pxTcb = ( PeriodicTaskTCB_t * )pxParam;
    TickType_t xSkippedPeriod;

    while( 1 )
    {
        pxTcb->xJobRunning = true;
        vTraceAddRecord( pxTcb->xPeriodicTask.pcTaskName, PTL_EVT_START, pxTcb->ulReleaseNumber - pxTcb->ulOverrunCount );
        /* Execute user body */
        pxTcb->xPeriodicTask.xEntryFunction( pxTcb->xPeriodicTask.pxArgument );
        pxTcb->xJobRunning = false;

        TickType_t xFinishTime = xTaskGetTickCount(); /* F_k */

        if ( pxTcb->xJobSkipped )
        {
            /* Job skipped -> wake up next period and skip next release */
            xSkippedPeriod = ( pxTcb->ulOverrunCount + 1 ) * pxTcb->xPeriodicTask.xPeriod;
            vTraceAddRecord( pxTcb->xPeriodicTask.pcTaskName, PTL_EVT_END, pxTcb->ulReleaseNumber - pxTcb->ulOverrunCount );
            pxTcb->ulOverrunCount = 0;
            pxTcb->xJobSkipped = false;

            if ( !pxTcb->xJobMissed )
            {
                /* When job finished with no overrun, wait for next release */
                vTaskDelayUntil( &pxTcb->xReleaseTime, xSkippedPeriod );
            }
            else
            {
                /* If job was CATCH UP, start immediately the new one ( no vTaskDelayUntil() ) */
                pxTcb->xJobMissed = false;
                pxTcb->ulOverrunCount--;
            }
        }
        else if ( pxTcb->xJobMissed )
        {
            /* Job missed -> no delay and no release number increment */
        }
        else
        {
            /* Normal job execution -> check deadline and delay until next release */
            pxTcb->ulReleaseNumber++;
        }
    }
}
```




PTL.C [3]



```
static void prvTimerCallback( TimerHandle_t xTimer )
{
    /* 1. Retrieve which job called this callback with pvTimerGetTimerID */
    uint8_t *pucTaskIndex = ( uint8_t * ) pvTimerGetTimerID( xTimer );
    PeriodicTaskTCB_t *pxTcb = &xCurrentActiveTasks[ *pucTaskIndex ];

    /* 2. See if the job is currently still running */
    if ( pxTcb->xJobRunning )
    {
        /* 2.2 If yes, manage overrun (deadline miss seen in the wrapper) */
        switch ( xGlobalCfg.ePolicy )
        {
            /* case switch */
        }
    }
    else
    {
        vTraceAddRecord( /* release */ );
    }
}
```

```
case PTL_POLICY_SKIP:
    vTraceAddRecord( /* skip */ );
    pxTcb->xJobSkipped = true;
    pxTcb->ulReleaseNumber++;
    pxTcb->ulOverrunCount++;
    break;
case PTL_POLICY_KILL:
    vTaskDelete( pxTcb->xTaskHandle );
    xTaskCreate(
        prvPTLTaskWrapper,
        pxTcb->xPeriodicTask.pcTaskName,
        pxTcb->xPeriodicTask.usStackSize,
        ( void * )pxTcb,
        pxTcb->xPeriodicTask.xPriority,
        &pxTcb->xTaskHandle
    );
    pxTcb->xReleaseTime = xTaskGetTickCount();
    vTraceAddRecord( /* kill */ );
    pxTcb->ulReleaseNumber++;
    vTraceAddRecord( /* release */ );
    break;
case PTL_POLICY_CATCH_UP:
    pxTcb->xJobMissed = true;
    pxTcb->ulOverrunCount++;
    vTraceAddRecord( /* catch up */ );
    pxTcb->ulReleaseNumber++;
    vTraceAddRecord( /* release */ );
    pxTcb->xReleaseTime = xTaskGetTickCount();
    break;
```



TRACE.H



```
typedef enum
{
    PTL_EVT_RELEASE,
    PTL_EVT_START,
    PTL_EVT_END,
    PTL_EVT_DEADLINE_MISS,
    PTL_EVT_JOB_MISS,
    PTL_EVT_OVERRUN_SKIP,
    PTL_EVT_OVERRUN_KILL,
    PTL_EVT_OVERRUN_CATCH_UP
} TraceEvent_t;

typedef struct
{
    TickType_t xTimeEvent;      /* When did the event occur? */
    const char *pcTaskName;    /* What task? */
    TraceEvent_t eEvent;       /* What did it do? */
    uint32_t ulReleaseNumber;   /* Which instance? */
} TraceRecord_t;

/* Kernel hooks to detect IDLE_TASK slices */
void vTraceTaskSwitchedIn( void )
{
    if ( xTaskGetCurrentTaskHandle() == xTaskGetIdleTaskHandle() )
        vTraceAddRecord( "IDLE_TASK", PTL_EVT_START, 0 );
}

void vTraceTaskSwitchedOut( void )
{
    if ( xTaskGetCurrentTaskHandle() == xTaskGetIdleTaskHandle() )
        vTraceAddRecord( "IDLE_TASK", PTL_EVT_END, 0 );
}
```

IN FREERTOSCONFIG.H:

```
/* KERNEL TRACE HOOKS */
extern void vTraceTaskSwitchedIn( void );
extern void vTraceTaskSwitchedOut( void );
#define traceTASK_SWITCHED_IN()    vTraceTaskSwitchedIn()
#define traceTASK_SWITCHED_OUT()   vTraceTaskSwitchedOut()
```



TRACE.C [1]



```
void vTraceAddRecord( const char *pcTaskName, TraceEvent_t eEvent, uint32_t ulReleaseNumber )
{
    TickType_t xCurrentTime;

    if( ulCount < traceBUFFER_SIZE )
    {
        xCurrentTime = xTaskGetTickCountFromISR();
        xTraceBuf[ ulCurrentIndex ].xTimeEvent = xCurrentTime;
        xTraceBuf[ ulCurrentIndex ].pcTaskName = pcTaskName;
        xTraceBuf[ ulCurrentIndex ].eEvent = eEvent;
        xTraceBuf[ ulCurrentIndex ].ulReleaseNumber = ulReleaseNumber;

        ulCurrentIndex++;
        ulCount++;
    }

    static const char* prvEventToString( TraceEvent_t eEvent )
    {
        switch ( eEvent )
        {
            case PTL_EVT_RELEASE:      return "RELEASE";
            case PTL_EVT_START:        return "START";
            case PTL_EVT_END:          return "END";
            case PTL_EVT_DEADLINE_MISS: return "END WITH DEADLINE MISS";
            case PTL_EVT_JOB_MISS:     return "END (MISSED)";
            case PTL_EVT_OVERRUN_SKIP: return "OVERRUN + SKIP";
            case PTL_EVT_OVERRUN_KILL: return "OVERRUN + KILL";
            case PTL_EVT_OVERRUN_CATCH_UP: return "OVERRUN + CATCH_UP";
            default:                   return "UNKNOWN";
        }
    }
}
```



TRACE.C [2]



```
void vTraceDump( void )
{
    char cUartBuffer[ 50 ];
    TraceRecord_t xSnapBuf[ traceBUFFER_SIZE ]; /* Snapshot of the buffer at calling time */
    uint32_t ulTempIndex;

    for ( ulTempIndex = 0; ulTempIndex < ulCount; ulTempIndex++ )
        xSnapBuf[ ulTempIndex ] = xTraceBuf[ ulTempIndex ];

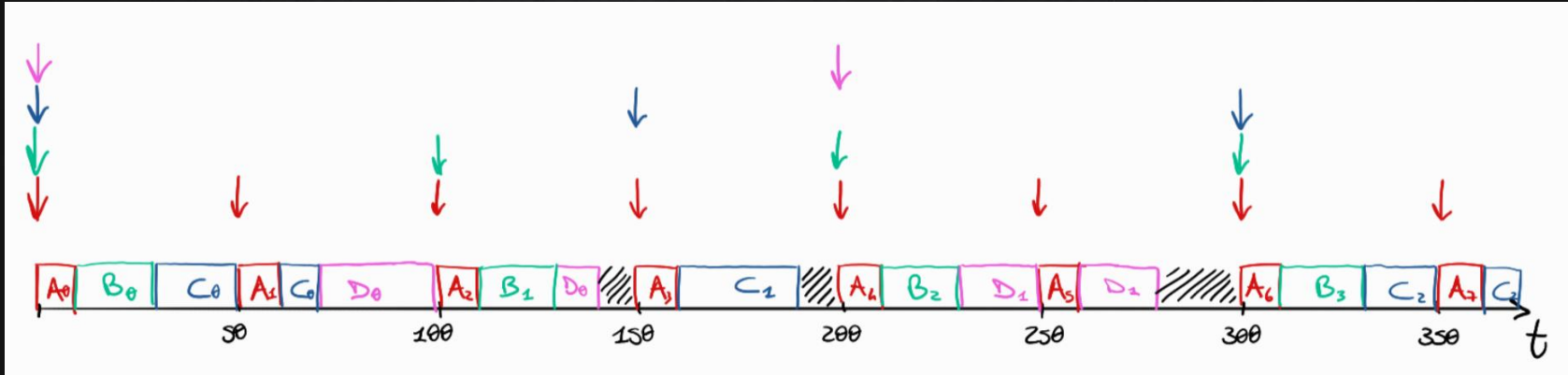
    // Print on screen the content of the snapshot using UART
    for ( ulTempIndex = 0; ulTempIndex < ulCount; ulTempIndex++ )
    {
        if ( strcmp( xSnapBuf[ ulTempIndex ].pcTaskName, "IDLE_TASK" ) == 0 )
        {
            sprintf( cUartBuffer, "[%4lu] %s %s\n", xSnapBuf[ ulTempIndex ].xTimeEvent,
                xSnapBuf[ ulTempIndex ].pcTaskName, prvEventToString( xSnapBuf[ ulTempIndex ].eEvent ) );
            UART_printf( cUartBuffer );
        }
        else
        {
            sprintf( cUartBuffer, "[%4lu] %s (%lu) %s\n", xSnapBuf[ ulTempIndex ].xTimeEvent, xSnapBuf[ ulTempIndex ].pcTaskName,
                xSnapBuf[ ulTempIndex ].ulReleaseNumber, prvEventToString( xSnapBuf[ ulTempIndex ].eEvent ) );
            UART_printf( cUartBuffer );
        }
    }
    ulCurrentIndex = 0;
    ulCount = 0;
}
```



TESTING BASED ON A GIVEN GOLDEN MODEL

- A → PRIORITY = 4, PERIOD = 50, WCET = 10
B → PRIORITY = 3, PERIOD = 100, WCET = 20
C → PRIORITY = 2, PERIOD = 150, WCET = 30
D → PRIORITY = 1, PERIOD = 200, WCET = 40

```
[ 0] task_A (0) RELEASE [130] task_B (1) END [280] task_D (1) END
[ 0] task_B (0) RELEASE [140] task_D (0) END [280] IDLE_TASK START
[ 0] task_C (0) RELEASE [140] IDLE_TASK START [300] IDLE_TASK END
[ 0] task_D (0) RELEASE [150] IDLE_TASK END [300] task_C (2) RELEASE
[ 0] task_A (0) START [150] task_C (1) RELEASE [300] task_B (3) RELEASE
[10] task_A (0) END [150] task_A (3) RELEASE [300] task_A (6) RELEASE
[10] task_B (0) START [160] task_A (3) END [300] task_A (6) START
[30] task_B (0) END [160] task_C (1) START [310] task_A (6) END
[30] task_C (0) START [190] task_C (1) END [310] task_B (3) START
[50] task_A (1) RELEASE [190] IDLE_TASK START [330] task_B (3) END
[50] task_A (1) START [200] IDLE_TASK END [330] task_C (2) START
[60] task_A (1) END [200] task_D (1) RELEASE [350] task_A (7) RELEASE
[70] task_C (0) END [200] task_A (4) RELEASE [350] task_A (7) START
[70] task_D (0) START [200] task_A (4) START [360] task_A (7) END
[100] task_B (1) RELEASE [210] task_B (2) START [370] task_C (2) END
[100] task_A (2) RELEASE [210] task_B (2) END
[100] task_A (2) START [230] task_B (2) END
[110] task_A (2) END [230] task_D (1) START
[110] task_B (1) START [250] task_A (5) RELEASE
[110] task_B (1) START [250] task_A (5) START
[110] task_B (1) START [260] task_A (5) END
```





THANK YOU FOR
YOUR ATTENTION



NOÈ GABRIELE - s307048
PAVANELLO MARCO - s361477
PITTALIS LUCA - s358755
RUSSO LORENZO - s358556

